Cardon Capstone Proposal

Capstone Project Proposal: Cardon

Janhavi Chavada · Turbin3 Builders Cohort · May 2026

Repo: github.com/Hijanhv/cardon

Project overview

Cardon is a transaction firewall for Solana AI agents. Every transaction an autonomous agent wants to broadcast first goes through Helius simulation, then a policy check against an on-chain Anchor program, and (when the policy flags it) sits in a queue for a human approver before it ever hits the network. Policies and approval rules live on-chain, not in middleware config, so a single operator can't quietly raise a spend cap or remove a kill switch. The goal is to let teams ship autonomous agents on Solana without one bad prompt, jailbreak, or model regression draining a hot wallet.

Part A: Final Project Proposal

1. Value Proposition and PMF

For teams shipping AI agents on Solana, Cardon is the policy enforcement and HITL approval layer that sits between an agent's intent and the network. You drop the Rust SDK (`cardon-rs`) or its TypeScript binding into any agent built on SendAI Agent Kit, Arc, or a custom Rig pipeline. You write your policies once into an Anchor program: per-asset spend caps, an allowlist of programs the agent can CPI into, time-of-day windows, rate limits, daily volume ceilings, anomaly thresholds. Anything the policy doesn't auto-approve gets pushed to a Next.js dashboard where authorized signers approve or reject before the agent broadcasts.

The three value areas this delivers:

1. **Loss containment that doesn't depend on the agent.** If the agent gets prompt-injected, jailbroken, or its underlying model regresses overnight, the policy still holds. The cap is enforced at the signing layer, not inside the agent's reasoning loop, so nothing the LLM does can bypass it.
2. **An audit log you can hand a compliance team.** Every approval, rejection, override, and policy change writes to an on-chain append-only log. When something goes wrong (and at some point it will), the post-mortem doesn't depend on whatever log file a single operator can or can't produce.
3. **Policy governance that survives operator churn.** The closest existing product, ClawdieLabs' Sentinel, ships as centralized middleware. Whoever has code access can change policies. Cardon puts the policy registry in an Anchor program with an owner authority that can be a multisig, a DAO, or a governance token. Rotating policy authority is an on-chain transaction, not a config push.

PMF hypothesis. SendAI Agent Kit has crossed 95,000 NPM downloads. Even at a 1% conversion to production-money agents, that's roughly 950 teams structurally exposed to the failure modes Cardon contains. Security infra historically sells reactively (Sentry-style error monitoring, Stripe Radar, Datadog), so the launch motion is to be the obvious default *before* the first big agent-driven loss makes urgency a market condition. The beachhead is the cohort of hackathon teams already shipping Solana agents (Latinum, Project Plutus, Agent Arc, Daiko, theintern.fun): they have the problem, no budget, and will adopt anything that integrates in one line and is free.

2. Target Markets

1. **Solana AI agent dev teams (primary).** Two-to-five-person teams shipping SendAI Agent Kit or Arc-based agents with live capital. Autonomous trading, DeFi rebalancing, treasury

optimization, social-graph automation. They have the firewall problem and the engineering capacity to integrate an SDK but not to build one from scratch.

2. **Hackathon teams shipping agent products (beachhead).** Cypherpunk and Breakout AI-track winners and follow-on cohorts. Free open-source SDK and a free hosted tier captures these as reference logos before any enterprise motion starts. They are the credibility flywheel.
3. **Solana protocols with agentic flows.** Drift, Jupiter, Kamino, Marginfi already integrate agentic execution. The protocols want to keep agent behavior inside their risk envelope without writing per-agent firewalls themselves. Cardon is the layer they recommend to integrating agent teams.
4. **DAOs and crypto-native treasury operators.** Groups that want to delegate execution to agents without trusting any single operator. The on-chain policy registry maps directly to multisig- or DAO-controlled agent fleets — rotating an operator out doesn't require trusting them to push a config change.
5. **Solana enterprise (secondary, not buying from a capstone).** Ledger, Magic Eden, Forvis Mazars-tier teams that are exploring agent automation but blocked by compliance and audit requirements. This matches Turbin3's existing customer base. They are not buying security infra from a 6-week cohort project, but the on-chain audit log is the unlock that gets them in the door after Cardon has shipped logos and incident-response credibility.

3. Competitor Landscape

Direct competitors

Competitor	What they ship	Where they're weak
ClawdieLabs Sentinel	Rust firewall, Helius simulation, HITL dashboard	Policy lives in centralized middleware. Any operator with code access can mutate it. No on-chain audit trail.
OpenZeppelin Defender	Ops platform with HITL approvals and policy gating. EVM-native.	Not Solana-native. No SVM account, CPI, or rent semantics. Expanding to non-EVM through 2026 but Solana isn't shipped yet.
Hypernative / Forta-on-Solana	Real-time anomaly detection on Solana transactions	Alerting, not prevention. No interception path. By the time the alert fires, the transaction has confirmed.

Adjacent and partial overlap

Player	Overlap	Why it doesn't fully solve it
Squads Multisig	Human-driven approval workflows	Built for low-throughput human multisig, not for the volume an agent generates. Cardon's HITL queue is a natural Squads integration, not a Squads replacement.
Helius webhooks + custom code	Tx-level monitoring and reaction	DIY. No policy primitives, no HITL UX, no audit trail. Every team rebuilds the same plumbing.
Código.ai	Solana-tuned LLM for code audit	Audits the agent's program code statically. Doesn't see runtime behavior or transaction intent.
OtterSec / Kani formal verification	Mathematical correctness proofs	Pre-deployment static. Complementary to a runtime firewall, not competitive.
SendAI internal policy hooks	Action-level allow/deny inside the kit	Library-level. Mutable by anyone with code access. Not portable across agents or wallets.

What the AI missed and I had to add manually

- **Squads.** The AI didn't surface it at all in its initial competitor list. This is the most important manual add: Squads is the canonical Solana multisig primitive and the obvious *integration* target for Cardon's HITL queue. Treating it as a competitor was the wrong frame; integrating with it from week 1 is the right one.
- **OpenZeppelin Defender, weighted properly.** The AI named Defender but treated it as roughly equivalent to Sentinel. It's not — Defender is a much bigger company with an active non-EVM expansion roadmap. If they ship Solana support before Cardon mainnet, the only moat left is SVM-native policy primitives (rent-aware caps, CPI-depth limits, compute-budget gating). That's a real risk to plan around, not a feature parity issue.
- **SendAI absorbing the category.** The AI missed that SendAI could ship a built-in policy layer in the Agent Kit itself, which would compress Cardon's wedge to nothing for SendAI-only users. The counter is to ship Cardon as a SendAI plugin from day one (their `packages/adapter-*` interface is already documented) so that the *official* way to add policy hooks is via Cardon, not against it.
- **Hackathon winners as competitors vs. customers.** Every AI-track Cypherpunk and Breakout winner I looked at is a Cardon integration candidate, not a competitor. The AI didn't

make this distinction — it lumped them in as competitive overlap when they're actually the beachhead customer list.

4. Founder-Market Fit

Background. I'm a builder in the Turbin3 Builders cohort and Cardon is my capstone. The cohort is the 6-week Rust + Anchor track — the deliverable expected from me is a production-grade Solana program, not a tutorial-shaped toy. The repo at github.com/Hijanhv/cardon is the proof-in-progress: the Anchor program compiles, the SDK mirrors the on-chain policy logic, and the Next.js dashboard reads pending approvals from the program via `getProgramAccounts` with a memcmp filter.

Skills. The relevant ones for shipping Cardon are baseline Rust comfort, Anchor program structure (PDAs, account constraints, CPI signing, rent semantics), and enough TypeScript to wire a dashboard against an Anchor IDL. I'm not coming in claiming to be a senior security engineer — I'm coming in as a builder who's chosen to ship into a category I want to learn deeply, and I'm using the cohort's mentor access and deadline structure to force the learning curve into a 6-week window.

Passion. What pulled me into this category specifically is that AI agents that can sign Solana transactions create a brand-new failure mode, and the infrastructure to contain it doesn't exist yet. There's no obvious incumbent, the category is empirically getting harder (more agents shipping every quarter), and the existing precedent (Sentinel) ships as centralized middleware that anyone with code access can mutate. Building the on-chain policy primitive for this is the kind of greenfield problem I want to ship into — it sits at the intersection of crypto-native primitives (multisigs, on-chain governance, append-only logs) and the AI-agent failure mode the rest of the industry is still pretending isn't real.

Network. Every Solana agent team I'd want as an early integration partner is one warm intro away inside the Turbin3 alumni graph. Alumni are placed at Anagram, Paladin, Metaplex, Drift, Compute Labs, and across the 200+ partner protocols Turbin3 lists publicly. The motion for getting the first 5 pilot teams is "ask in the cohort Slack and the alumni channel," not cold outbound. That's a real distribution advantage I get specifically because I'm in this cohort, not despite it.

Honest weakness. I don't have a published security-research history. B2D infra sales to sophisticated agent operators don't close on marketing or a pitch deck — they close on a real incident, a credible response, and a track record over months. The counter-motion available to a single capstone builder is to open-source the SDK from day one, run a free hosted tier, get hackathon teams as reference integrations through the cohort and Colosseum networks, and then earn enterprise trust after Cardon has caught its first real attack in production. Enterprise sales come after the credibility, not as the launch motion.

Part B: Process Appendix

B.1 Starting point

I came into this assignment knowing two things: I want to ship a real Solana program as my Turbin3 capstone, and I want it to be something the cohort itself takes seriously when grading. I did not have a specific product idea on day one. The work below is how I got from "Anchor program, real product, 6 weeks" to Cardon.

B.2 Discovery research (before any AI prompting)

Before prompting any AI about value props or competitors, I scraped and read the actual primary sources, so the AI outputs in B.3 would have something honest to compare against.

Sources I went through manually (all archived under `/research/`):

- `turbin3.org/` — Turbin3 is the Solana Talent Engine / Web3 Builders Alliance education arm. 2,000+ devs trained, 200+ hiring partners (Jupiter, Wormhole, Metaplex, Solana Foundation, Drift, Marginfi, Helius, Ledger, Anagram).
- `turbin3.org/institute` — Programs: Builders (flagship, 6 weeks, Rust + Anchor + capstone), Accelerated Builders, Pinocchians Working Group, SVM, plus a coming module named "Solana Model and Agent Verification."
- `turbin3.org/blog/ai-verification-security-solana` (May 8 2026, by CEO Nate Hughes) — this post is the entire thesis: AI agents on Solana need transaction firewalls, multi-stage verification, HITL pipelines. Cites Sentinel and Rig explicitly. Notes that generic LLMs have near-zero recall on Solana DeFi patterns.
- `turbin3.org/blog/ai-solana-svm-depth` (April 2026) — calls out that engineers need to "build and operate a multi-agent pipeline (architecture, coding, verification) against their own stack."
- `github.com/solana-turbin3` — past cohort capstones (Q1_25, Q2_25): BloodLedger, Payclip (USDC payment links), AMM, escrow, dice, NFT staking, Token-2022 vault. Useful as a calibration of what scope past cohorts have shipped.
- **Solana Cypherpunk Hackathon winners (Dec 2025).** Stablecoin track winner: MCPay. Honorable mentions included Mercantill (enterprise banking for AI agents), Sentinel Agent, Seeker OS.
- **Solana Breakout Hackathon winners (Jul 2025).** AI track top five: Latinum (payment middleware for MCP builders), Project Plutus (deploy any AI agent on Solana), Agent Arc (non-custodial AI trading terminal), Daiko, theintern.fun. University track winner: Synto.

What this gave me. Turbin3's published thesis and the live hackathon meta were saying the same thing from two angles. The category is real, the buyer pool exists, and Turbin3's cohort judging is pre-aligned with this space. So my capstone idea narrowed to "build into the firewall /

policy / verification layer Turbin3 is publicly saying needs to exist." That's where Cardon comes from.

B.3 AI prompts and synthesized outputs

B.3.1 Value prop prompt

> *"Based on this idea (on-chain Solana transaction firewall for AI agents that intercepts transactions, checks them against an on-chain policy program, and routes high-risk transactions to a human approver), help outline the core value proposition and 2-3 key value areas. What is the initial PMF case?"*

Synthesized AI output:

1. Containment of autonomous capital — the firewall caps the worst-case loss from a bad prompt or jailbreak.
2. Compliance and audit — enterprise agent deployment is blocked without provable controls and a verifiable trail.
3. Decentralized policy governance — Sentinel is centralized; an on-chain policy registry is differentiated.

My take on this output. It's directionally right but value area 3 is overstated as written. "Decentralized governance" is only valuable if buyers care, and not all of them do — many agent teams will prefer fast-iteration centralized policy controls. I noted this for the adversarial critique in B.4 and refined it in B.5.

B.3.2 Target markets prompt

> *"For an on-chain transaction firewall for Solana AI agents with the value prop above, suggest 2-5 target demographics or market segments. Which is the right beachhead?"*

Synthesized AI output: SendAI / Arc developer teams, Solana protocols with agentic flows, DAO and treasury operators, enterprise teams (Ledger / Magic Eden / Forvis Mazars), hackathon agent teams. Beachhead suggestion: enterprise, because they have budget.

My take. The AI got the *list* mostly right but the *beachhead suggestion* was wrong, which I caught against the Cypherpunk and Breakout winner lists I had already read. Enterprise can't actually buy security infra from a 6-week capstone build — there's no track record, no SOC2, no insurance posture. Hackathon teams will. They're the only segment that can move fast enough to be reference logos in the capstone timeline. I demoted enterprise to secondary and promoted hackathon teams to beachhead in B.5.

B.3.3 Competitor prompt

> *"Identify competitors for an on-chain transaction firewall for Solana AI agents targeting agent dev teams, protocols, DAOs, and enterprise. What are the gaps in their offerings?"*

AI-identified competitors: ClawdieLabs Sentinel, OpenZeppelin Defender, Hypernative / Forta.

Competitors I found through manual research (AI missed):

- **Squads Multisig.** Most important miss. Squads is the canonical Solana multisig and is the obvious integration target for Cardon's HITL queue, not a competitor. The AI didn't surface it at all.
- **Código.ai.** Adjacent verification (audits Solana programs, not runtime behavior). Worth listing for completeness because a buyer comparing security tooling will see them and ask the difference.
- **SendAI internal policy hooks.** The AI didn't realize SendAI's own kit has a basic `actions` interface that includes light allow/deny logic. This is the absorb-the-category risk, and the response is to ship Cardon as a SendAI plugin from day one rather than against it.
- **OtterSec / Kani formal verification.** Pre-deployment proof tooling. Not a runtime firewall, but a buyer will still ask the difference.

Gap analysis. The AI biased toward EVM analogs (Defender) and missed Solana-native primitives (Squads especially, SendAI internals). The highest-signal source for the manual sweep was the Cypherpunk and Breakout winner lists, because reading them made it obvious that almost every "AI-track" project is actually a *customer* of Cardon, not a competitor — a distinction the AI didn't draw on its own.

B.3.4 FMF prompt

> *"Given that I'm a Turbin3 Builders cohort student building this as my capstone, with the network access and cohort scope match that implies, how should I frame founder-market fit for this idea? What are the honest weaknesses I should not paper over?"*

Synthesized AI output: Frame strengths around timing match (Turbin3 thesis), distribution match (alumni network), and scope match (6-week Anchor capstone). Honest weakness is no prior security research history; counter-motion is open-source + free tier + hackathon-team reference logos.

My take. Mostly right. I used this framing in §4 but with one edit — I removed any phrasing that overclaimed "deep" experience and let the FMF stand on the verifiable structural fit (timing, network, scope) rather than on credentials I don't have.

B.4 Adversarial AI critique

> *"Critique this project's value prop, target market, and competitive analysis (Cardon, on-chain firewall for Solana AI agents). Why might this not be a true blue ocean? What types of competitors might have been overlooked?"*

Synthesized critique:

1. **"Blue ocean" is overstated.** Sentinel already exists. The on-chain policy registry is one design choice, not a moat unless buyers actually care about decentralizing policy. Many will prefer centralized controls for speed.
2. **Buyer-behavior risk.** Agent dev teams are not security buyers by default. They buy yield and speed. Selling firewalls pre-incident is a hard sell.
3. **Squads cannibalization.** If Squads ships an agent-wallet multisig primitive, Cardon's HITL value collapses to "Squads + Helius simulation + a dashboard."
4. **Big-name distribution risk.** Helius, Jito, or Anza could ship a first-party agent firewall as an RPC-level feature and end the market.
5. **Code-fork risk.** Anyone can fork an Anchor program. The real moat is the live policy graph and accumulated incident-response credibility, not the code.

My analysis of the critique's validity. Points 1, 2, 3, and 5 are valid and forced real changes. Point 4 is plausible but hypothetical — Helius and Anza don't currently ship policy/HITL infra and Jito's focus is MEV, so I noted it as a watch-out but didn't restructure around it.

B.5 Refinements and rationale

Value-prop refinement. Removed any "blue ocean" framing. Reframed value area 3 from "decentralized governance is the wedge" to "policy governance that survives operator churn, when the buyer cares about that property." Acknowledged in the same paragraph that Sentinel is a real precedent. *Reason:* the critique was correct that firewall-as-feature isn't a moat. The honest framing is that governance is a *differentiator for a subset of buyers*, not a universal moat. Stating that explicitly is more credible than overclaiming.

Target-market refinement. Demoted enterprise from primary to secondary. Promoted hackathon teams from "also a market" to explicit beachhead. *Reason:* the buyer-behavior critique is sharp. A 6-week capstone cannot close enterprise. It can win hackathon teams in week 6. The credibility flywheel has to start with the segment that can actually adopt fast.

Competitor refinement. Added Squads (reframed as integration target, not competitor), Código, SendAI internal hooks, OtterSec / Kani. *Reason:* the most dangerous "competitor" is not another firewall — it's the protocol-layer absorb move from Squads, Helius, or SendAI. The response is to ship integrations *with* the most likely absorbers (SendAI plugin, Squads-backed HITL) from week 1, so that when the absorb question comes up the answer is "Cardon is already how SendAI does this."

B.6 FMF critique and refinement

AI critique of the FMF:

> *"A capstone-builder claiming security infra credibility is structurally thin. The B2D motion needs incident-response evidence a single capstone cannot fully demonstrate. What is the honest version that doesn't overclaim?"*

My response. Valid. The original draft of the FMF leaned on "experienced Web3 builder" framing that wasn't honest about my actual stage. I rewrote it into the four explicit beats the rubric asks for — *Background, Skills, Passion, Network* — and added an explicit *Honest weakness* paragraph. Each beat is grounded in something verifiable: the cohort I'm in, the program that already compiles in the repo, the named alumni placements, and the published Turbin3 thesis post. I deliberately did not claim a security-research history I don't have. The honest framing is more credible to a security-infra-aware reader than the overclaim version, because anyone evaluating this seriously will see through overclaim immediately and discount everything else that follows.

Appendix C: Sources

- Turbin3 home, institute, blog, success stories, FAQ: https://turbin3.org/
- Turbin3 May 2026 thesis post: https://turbin3.org/blog/ai-verification-security-solana
- Turbin3 April 2026 SVM-depth post: https://turbin3.org/blog/ai-solana-svm-depth
- Past cohort repos: https://github.com/solana-turbin3
- Solana Cypherpunk Hackathon winners (Dec 2025): https://blog.colosseum.com/announcing-the-winners-of-the-solana-cypherpunk-hackathon/
- Solana Breakout Hackathon winners (Jul 2025): https://blog.colosseum.com/announcing-the-winners-of-the-solana-breakout-hackathon/
- SendAI Solana Agent Kit: https://github.com/sendaifun/solana-agent-kit
- Anchor framework: https://www.anchor-lang.com/
- Solana Foundation templates (`nextjs-anchor`, `react-vite-anchor`, `pinocchio-counter`): https://solana.com/developers/templates

Raw scrapes of the primary sources are in `/research/` in the project repo: https://github.com/Hijanhv/cardon